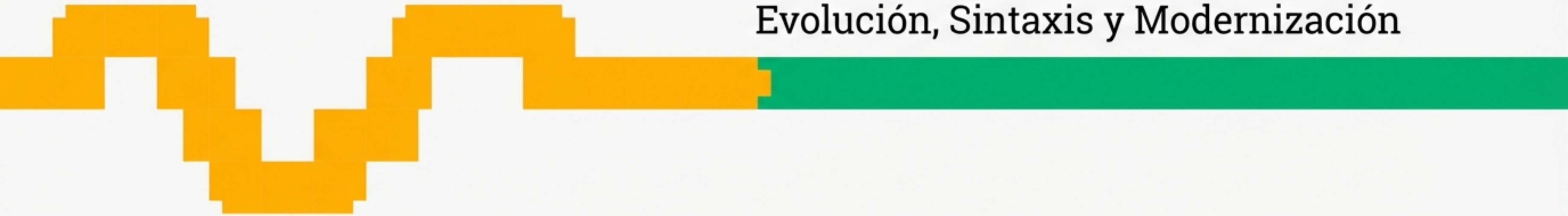


Python a través del tiempo

Evolución, Sintaxis y Modernización



Un análisis forense del cambio: De la ambigüedad de Python 2 a la claridad de Python 3.

Una guía visual sobre cómo el lenguaje ha priorizado cuatro pilares fundamentales. Basado en un análisis de fragmentos de código, compatibilidad y refactorización.

La Filosofía del Cambio

Python 3 no fue solo una actualización; fue una corrección de rumbo para eliminar redundancias y peligros.



Seguridad

Eliminación de evaluaciones de código implícitas (ej. `input` en Py2).

```
input("What is your name?")
```



Claridad

Operaciones explícitas en lugar de mágicas (ej. división real vs. entera).

```
10 / 4  
10 // 4
```



Expresividad

Nuevas sintaxis para escribir menos y hacer más (f-strings, pattern matching).

```
f"Hello, {name}!"  
match command:  
    case "quit":
```



Consistencia

Unificación del manejo de texto (Unicode por defecto) y funciones del núcleo.

```
s = "texto unicode"  
print("hello")
```


Fragmento A: La transición de Instrucción a Función

File Card

PYTHON 2 - INSTRUCCIÓN

```
print "Hola mundo"
```

File Card

PYTHON 3 - FUNCIÓN

```
print("Hola mundo")
```

Python 2: 'print' es una instrucción.
No requiere paréntesis.

Python 3: 'print()' es una función.
Los paréntesis son obligatorios.

Why it matters: Al ser una función, permite mayor flexibilidad (argumentos sep/end) y consistencia con el lenguaje.

Fragmento B: La Trampa de la Entrada de Datos

```
numero = input("Introduce un número: ")  
resultado = numero + 5
```

Python 2 (Riesgo)

input() evalúa entrada como CÓDIGO.
Vulnerabilidad de seguridad crítica.

Python 3 (Seguro)

input() devuelve STRING. Sumar + 5
provoca TypeError. Obliga a ser explícito.

Python 3 elimina la ambigüedad entre `raw_input()` e `input()`, priorizando la seguridad sobre la conveniencia.

Fragmento C: El Cisma de la División

```
print(5 / 2)
```

Legacy (Python 2)

Modern (Python 3)

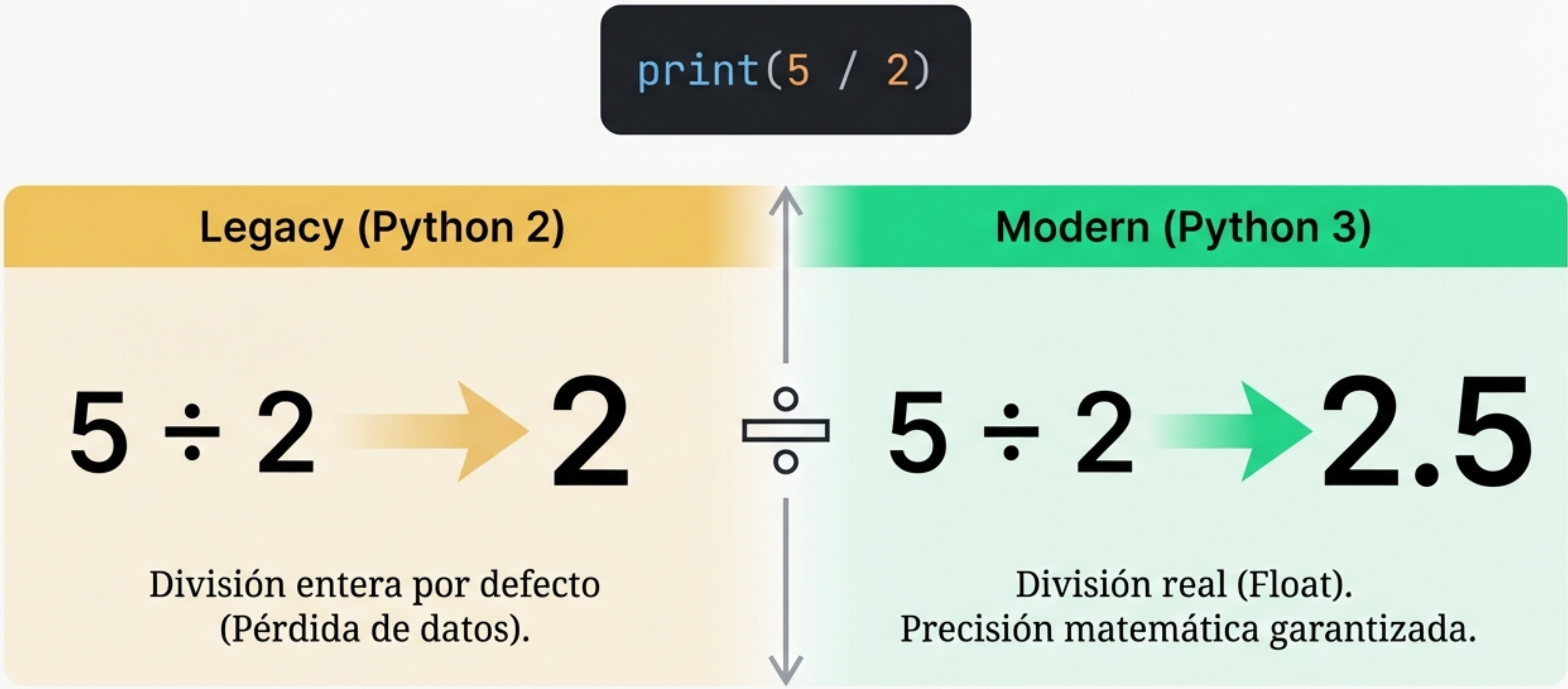
$5 \div 2 \rightarrow 2$

División entera por defecto
(Pérdida de datos).

$\div$

$5 \div 2 \rightarrow 2.5$

División real (Float).
Precisión matemática garantizada.

Para obtener división entera en Python 3, use el operador '//'.


Fragmento D: Expresividad Moderna (f-strings)

Introducido en Python 3.6

Antes (Complejo)

```
nombre = "Ana"  
print("Hola %s" % nombre)  
# o  
print("Hola {}".format(nombre))
```

Ahora (f-strings)

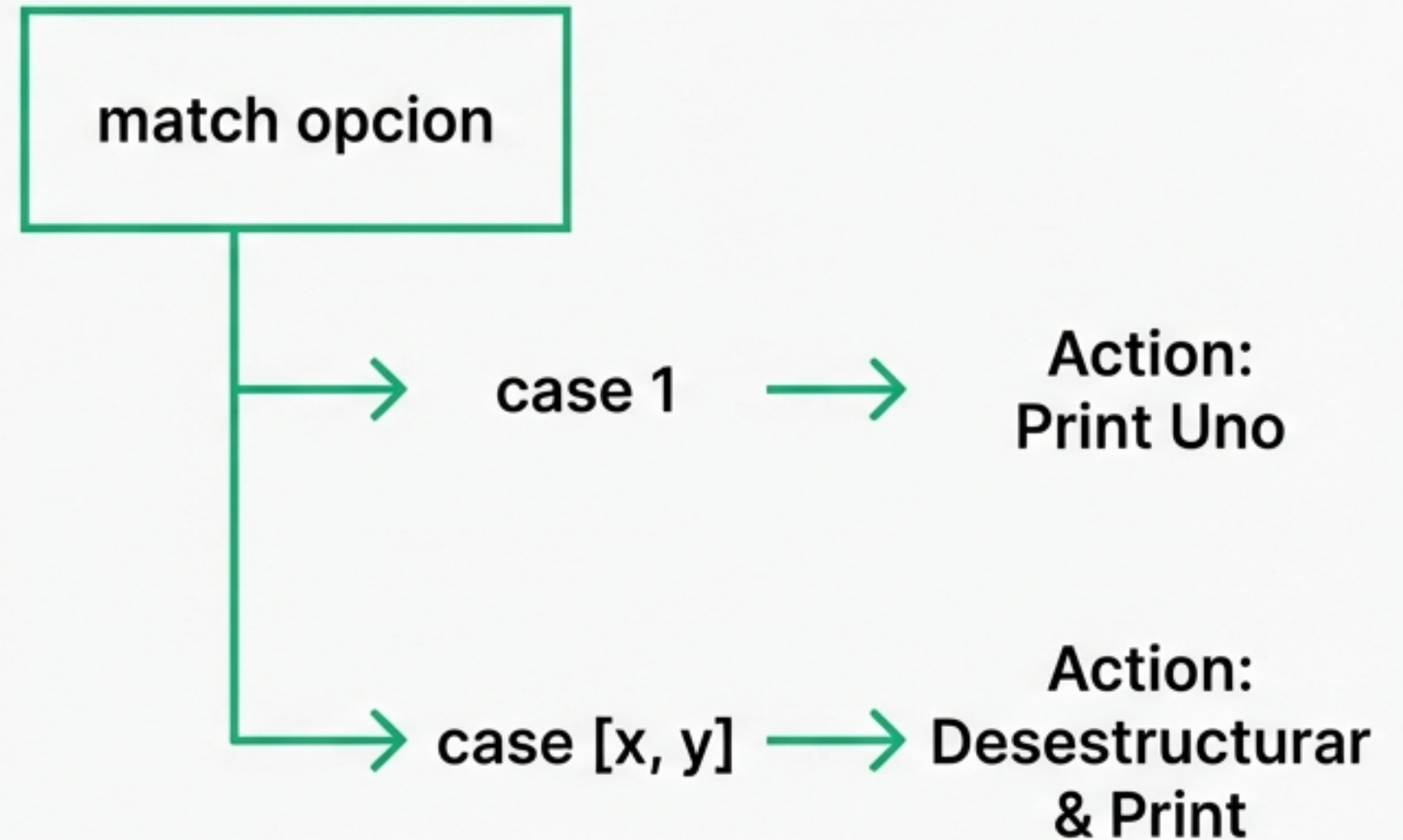
```
nombre = "Ana"  
print(f"Hola {nombre}")
```

- Más legible
- Más conciso
- Ejecución más rápida

Fragmento E: Pattern Matching Estructural

Modern Control Flow (Python 3.10+)

```
match opcion:  
    case 1:  
        print("Has elegido uno")  
    case [x, y]:  
        print(f"Punto: {x}, {y}")
```



No es solo un "switch". Permite verificar tipos y desestructurar datos complejos en una sintaxis limpia.

La Gran División: Sintaxis y Lógica

Característica	Python 2 (Legacy)	Python 3 (Moderno)
Print	<pre>print "msg"</pre> (Instrucción)	<pre>print("msg")</pre> (Función)
Input	<pre>input()</pre> evalúa código (Peligro)	<pre>input()</pre> siempre retorna string
Soporte	EOL 2020 (Sin seguridad)	Activo y Seguro

La Gran División: Datos y Matemáticas

Característica	Python 2 (Legacy)	Python 3 (Moderno)
División (/)	5/2 = 2 (Entera oculta)	5/2 = 2.5 (Float real)
Texto (Encoding)	ASCII por defecto / Bytes. Requiere <code>`u"...`</code>	Unicode por defecto. Soporte universal.

El cambio a Unicode por defecto eliminó una clase entera de errores de codificación, facilitando la internacionalización de aplicaciones modernas.

Comprobación del Entorno Real

```
$ python --version  
Python 3.11.4
```

✓ Soporte f-strings (3.6+)

✓ Soporte match (3.10+)

Simulación



¿Ejecutando código antiguo aquí?

- `print 'test'` -> **SyntaxError**
- `raw_input()` -> **NameError**
- `5/2` -> `2.5` (Cambio lógico silencioso)

Auditoría de Código Legacy (Python 2)

```
print "Bienvenido"
```

← Error de Sintaxis:
Faltan paréntesis

```
nombre = raw_input("Nombre: ")
```

```
edad = input("Edad: ")
```

← Función obsoleta

← RIESGO: Evalúa código / Tipo implícito

```
if edad >= 18:
```

```
    print "Mayor"
```


Migración: Sintaxis y Tipado Seguro

Paso 1: Corrección de Errores

```
$ python 3
```

```
print("Bienvenido")
```

Paréntesis añadidos

```
nombre = input("Nombre: ")
```

Reemplazo de raw_input

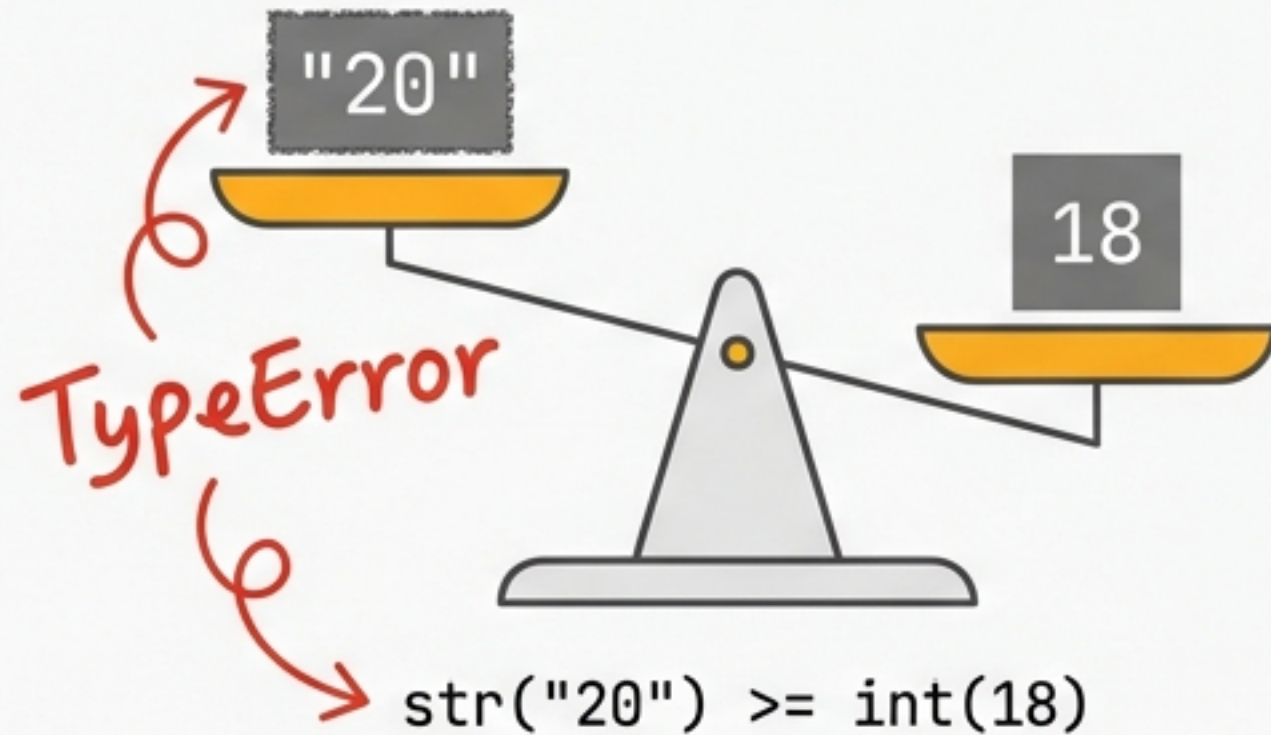
```
edad = int(input("Edad: "))
```

Conversión explícita
requerida (String -> Int)

```
if edad >= 18:  
    print("Mayor")
```

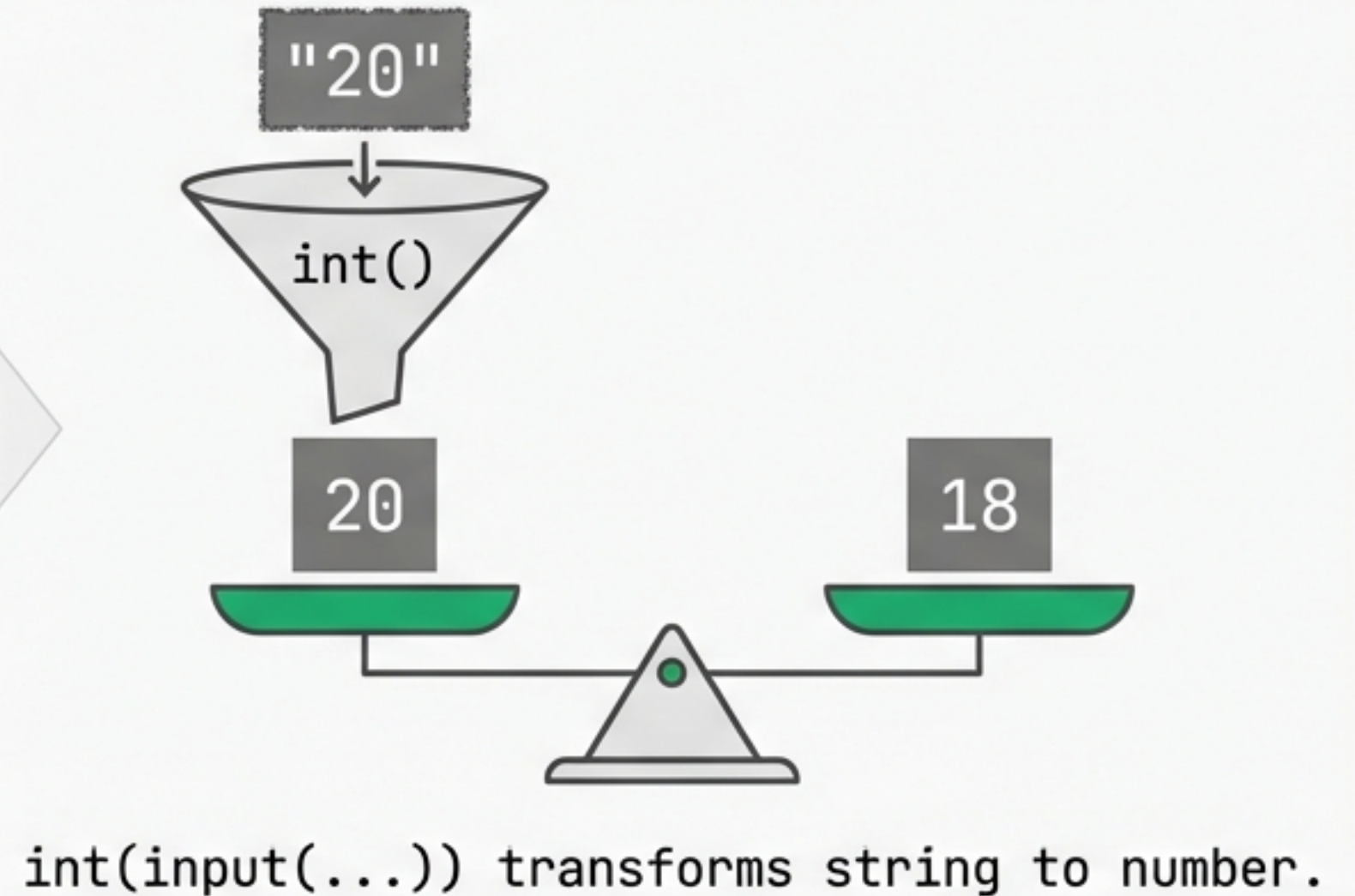

El Foco en la Lógica: La Conversión Explícita

El Problema



El programador debe declarar su intención de tratar el dato como número.

La Solución



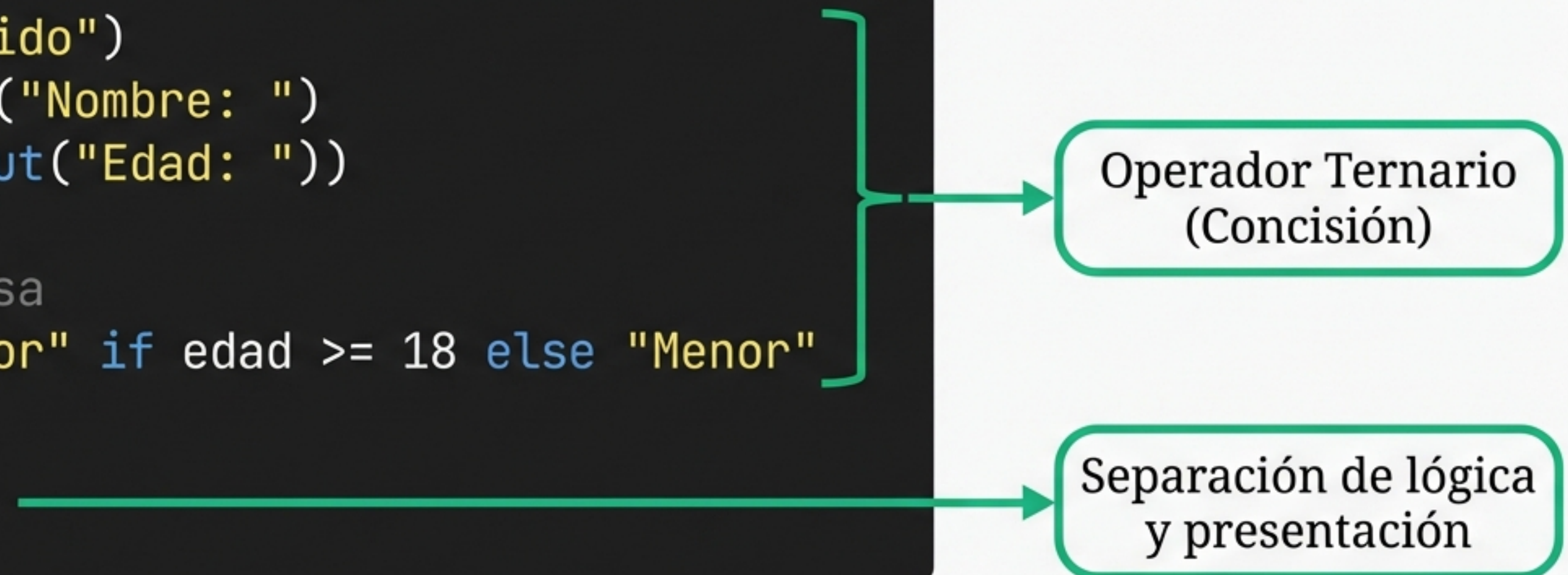
Claridad: El programador debe declarar su intención de tratar el dato como número.

Modernización Avanzada (Refactorización)

```
print("Bienvenido")
nombre = input("Nombre: ")
edad = int(input("Edad: "))

# Lógica concisa
mensaje = "Mayor" if edad >= 18 else "Menor"

print(mensaje)
```



Operador Ternario
(Concisión)

Separación de lógica
y presentación

Resultado: Código más limpio, fácil de mantener y menos propenso a errores.

Conclusión: Un Lenguaje Maduro

Seguridad: `input()` ya no ejecuta código arbitrario.

Claridad: La división y los tipos son explícitos, reduciendo bugs.

Expresividad: Herramientas como f-strings simplifican la lectura.

Consistencia: Unicode nativo resuelve la deuda técnica.

Todo desarrollo nuevo debe realizarse en Python 3. Mantenerse en el pasado es renunciar a la seguridad y la eficiencia.